

Universidade do Vale do Itajaí

Escola Politécnica

Banco de Dados

PROJETO DE BANCO DE DADOS

SISTEMA DE MENSAGERIA INSTANTÂNEA

Autor: Artur Nascimento Freiburger

Trabalho apresentado à disciplina de Banco de Dados, do curso de Ciências da Computação, como requisito de avaliação da terceira etapa.

Professor: Maurício Freitas

Itajaí

2026

SUMÁRIO

Este trabalho apresenta o desenvolvimento de um projeto de banco de dados aplicado ao domínio de mensageria instantânea. O sistema proposto foi inspirado em aplicativos como WhatsApp e Telegram e tem como objetivo permitir o cadastro de usuários, a criação de conversas, o gerenciamento de participantes e o envio de mensagens.

O projeto contempla as etapas de modelagem conceitual, lógica e física do banco de dados. No projeto conceitual, são apresentadas as entidades Usuário, Conversa, Participante e Mensagem, bem como seus atributos, relacionamentos e cardinalidades. No projeto lógico, essas entidades são transformadas em tabelas relacionais normalizadas, com definição de chaves primárias, chaves estrangeiras e restrições de integridade.

No projeto físico, é apresentada a implementação do banco de dados utilizando MySQL, incluindo os comandos SQL para criação do esquema, das tabelas, dos relacionamentos, dos índices e dos dados de exemplo. Também foi desenvolvida uma aplicação em Java, utilizando JDBC e interface por linha de comando, responsável por realizar operações de cadastro, consulta, atualização e remoção de usuários, conversas e mensagens.

A aplicação utiliza persistência em banco de dados relacional e busca demonstrar a integração entre a modelagem de dados e a implementação de um sistema funcional. O projeto evidencia a aplicação de conceitos como normalização, integridade referencial, relacionamento muitos-para-muitos, utilização de chaves compostas e execução de operações CRUD.

DEFINIÇÃO

Os serviços de mensageria instantânea fazem parte da rotina de pessoas, empresas e instituições que precisam trocar informações de maneira rápida e organizada. Aplicativos como WhatsApp e Telegram permitem que usuários criem conversas privadas ou em grupo, adicionem participantes e enviem mensagens. Para que esse tipo de serviço funcione corretamente, é necessário armazenar dados sobre os usuários cadastrados, as conversas existentes, os participantes de cada conversa e as mensagens enviadas.

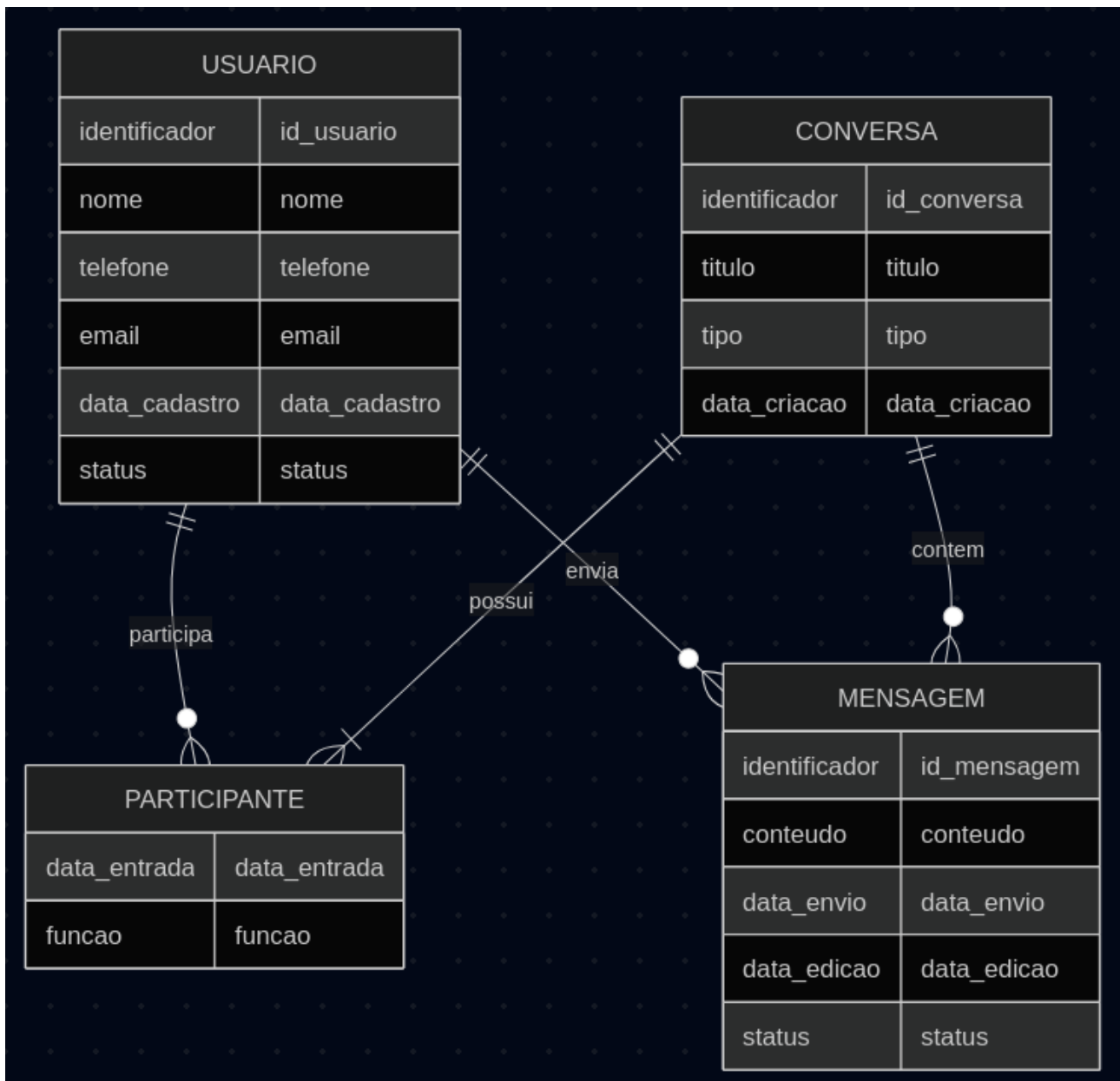
O problema abordado neste projeto é a necessidade de organizar essas informações em um banco de dados relacional, garantindo que os dados sejam armazenados de forma consistente e que os relacionamentos entre eles sejam preservados. Sem uma modelagem adequada, poderiam ocorrer problemas como duplicidade de usuários, mensagens associadas à conversa errada, participação repetida de uma mesma pessoa em um grupo ou envio de mensagens por usuários que não pertencem à conversa.

O sistema proposto é destinado a usuários que desejam se comunicar por meio de mensagens de texto em conversas privadas ou em grupo. Para fins acadêmicos, foi desenvolvida uma versão simplificada de um aplicativo de mensageria, concentrada nas operações necessárias para demonstrar a modelagem e a implementação de um banco de dados relacional.

A solução permite cadastrar usuários, criar conversas, adicionar participantes, enviar mensagens, consultar históricos e realizar alterações ou remoções nos registros. O sistema utiliza Java para a aplicação, JDBC para a comunicação com o banco de dados e MySQL como sistema gerenciador de banco de dados.

O principal objetivo é demonstrar a integração entre o projeto conceitual, o projeto lógico normalizado, o projeto físico e uma implementação funcional. Dessa forma, o trabalho evidencia a aplicação de conceitos como entidades, relacionamentos, cardinalidades, chaves primárias, chaves estrangeiras, restrições de integridade, normalização e operações CRUD.

Projeto Conceitual



Fonte: Imagem elaborada pelo autor utilizando mermaid.live

PROJETO LÓGICO

O projeto lógico transforma as entidades e os relacionamentos definidos no modelo conceitual em estruturas relacionais. Nessa etapa, são determinadas as tabelas, os atributos, as chaves primárias, as chaves estrangeiras e as restrições necessárias para preservar a integridade dos dados.

O Sistema de Mensageria Instantânea foi organizado em quatro tabelas principais: **USUARIO**, **CONVERSA**, **PARTICIPANTE** e **MENSAGEM**. A tabela **PARTICIPANTE** funciona como uma tabela associativa, resolvendo o relacionamento muitos-para-muitos existente entre usuários e conversas.

Estrutura textual do modelo lógico

USUARIO

```
USUARIO (  
    id_usuario PK,  
    nome,  
    telefone UK,  
    email UK,  
    data_cadastro,  
    status  
)
```

A tabela **USUARIO** armazena os dados das pessoas cadastradas no sistema.

- **id_usuario**: identificador único do usuário e chave primária;
- **nome**: nome completo ou nome de exibição;
- **telefone**: número de telefone, com restrição de unicidade;
- **email**: endereço eletrônico, com restrição de unicidade quando informado;
- **data_cadastro**: data e hora do cadastro;
- **status**: indica se o usuário está ativo ou inativo.

CONVERSA

```
CONVERSA (  
    id_conversa PK,  
    titulo,  
    tipo,  
    data_criacao  
)
```

A tabela **CONVERSA** representa os espaços utilizados para a troca de mensagens.

- **id_conversa**: identificador único da conversa e chave primária;
- **titulo**: título da conversa, obrigatório para conversas em grupo;
- **tipo**: indica se a conversa é privada ou em grupo;
- **data_criacao**: data e hora da criação da conversa.

PARTICIPANTE

```

PARTICIPANTE (
    id_usuario PK, FK → USUARIO.id_usuario,
    id_conversa PK, FK → CONVERSA.id_conversa,
    data_entrada,
    funcao
)

```

A tabela PARTICIPANTE representa a participação de um usuário em uma conversa.

- **id_usuario**: identifica o usuário participante;
- **id_conversa**: identifica a conversa;
- **data_entrada**: data e hora em que o usuário entrou na conversa;
- **funcao**: identifica se o participante é membro ou administrador.

A chave primária da tabela é composta por **id_usuario** e **id_conversa**. Essa composição impede que um mesmo usuário seja inserido mais de uma vez na mesma conversa.

O atributo **id_usuario** também é uma chave estrangeira que referencia a tabela USUARIO. O atributo **id_conversa** é uma chave estrangeira que referencia a tabela CONVERSA.

MENSAGEM

```

MENSAGEM (
    id_mensagem PK,
    id_usuario,
    id_conversa,
    conteudo,
    data_envio,
    data_edicao,
    status,
    FK (id_usuario, id_conversa)
        → PARTICIPANTE (id_usuario, id_conversa)
)

```

A tabela MENSAGEM armazena as mensagens enviadas pelos usuários.

- **id_mensagem**: identificador único da mensagem e chave primária;

- `id_usuario`: identifica o autor da mensagem;
- `id_conversa`: identifica a conversa em que a mensagem foi enviada;
- `conteudo`: texto da mensagem;
- `data_envio`: data e hora do envio;
- `data_edicao`: data e hora da última edição;
- `status`: indica se a mensagem está enviada, editada ou removida.

A combinação dos atributos `id_usuario` e `id_conversa` forma uma chave estrangeira composta que referencia a tabela `PARTICIPANTE`. Essa restrição garante que um usuário somente possa enviar mensagens em uma conversa da qual participa.

Chaves do modelo

Chaves primárias

As chaves primárias utilizadas são:

- `USUARIO.id_usuario`;
- `CONVERSA.id_conversa`;
- `PARTICIPANTE.id_usuario` e `PARTICIPANTE.id_conversa`;
- `MENSAGEM.id_mensagem`.

A tabela `PARTICIPANTE` possui chave primária composta, formada pela combinação do usuário com a conversa.

Chaves estrangeiras

As chaves estrangeiras do modelo são:

- `PARTICIPANTE.id_usuario` referencia `USUARIO.id_usuario`;
- `PARTICIPANTE.id_conversa` referencia `CONVERSA.id_conversa`;
- `MENSAGEM.id_usuario` e `MENSAGEM.id_conversa` referenciam, em conjunto, a chave composta da tabela `PARTICIPANTE`.

Chaves únicas

A tabela `USUARIO` possui as seguintes restrições de unicidade:

- `telefone`;
- `email`.

Essas restrições impedem o cadastro de dois usuários com o mesmo telefone ou endereço de e-mail.

Relacionamentos do modelo lógico

O relacionamento entre USUARIO e PARTICIPANTE é do tipo um-para-muitos. Um usuário pode possuir nenhum ou vários registros de participação, enquanto cada participação pertence a exatamente um usuário.

USUARIO 1 — N PARTICIPANTE

O relacionamento entre CONVERSA e PARTICIPANTE também é do tipo um-para-muitos. Uma conversa pode possuir vários participantes, enquanto cada registro de participação pertence a uma única conversa.

CONVERSA 1 — N PARTICIPANTE

O relacionamento entre PARTICIPANTE e MENSAGEM é do tipo um-para-muitos. Um participante pode enviar nenhuma ou várias mensagens dentro da conversa, enquanto cada mensagem está associada a uma única participação.

PARTICIPANTE 1 — N MENSAGEM

Normalização

O modelo lógico foi normalizado até a Terceira Forma Normal.

Primeira Forma Normal

O modelo encontra-se na Primeira Forma Normal porque todas as tabelas possuem uma chave primária e seus atributos armazenam valores atômicos.

Não são utilizadas listas ou conjuntos de valores dentro de um único campo. Por exemplo, os participantes de uma conversa não são armazenados em uma lista na tabela CONVERSA. Cada participante possui um registro próprio na tabela PARTICIPANTE.

Segunda Forma Normal

O modelo encontra-se na Segunda Forma Normal porque todos os atributos não-chave dependem integralmente da chave primária de suas respectivas tabelas.

Na tabela PARTICIPANTE, os atributos `data_entrada` e `funcao` dependem da combinação entre o usuário e a conversa. Eles não dependem apenas do usuário nem apenas da conversa.

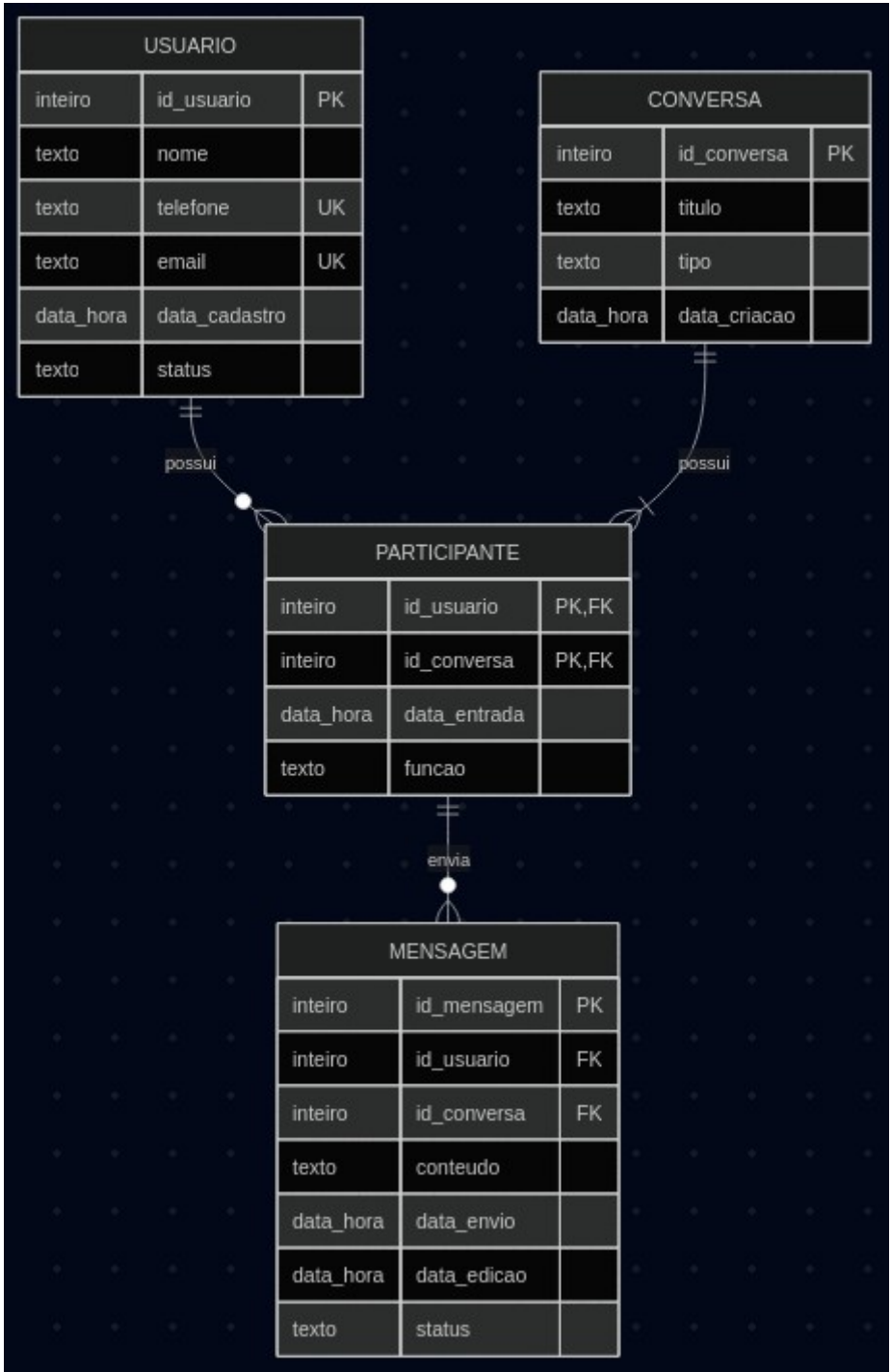
Terceira Forma Normal

O modelo encontra-se na Terceira Forma Normal porque não existem dependências transitivas entre atributos não-chave.

As informações de usuários são armazenadas somente na tabela USUARIO. Os dados das conversas são armazenados somente na tabela CONVERSA. A tabela MENSAGEM não repete informações como nome, telefone ou e-mail do autor, utilizando apenas as chaves necessárias para identificar o participante.

A normalização reduz a duplicidade de dados e evita anomalias durante as operações de inserção, atualização e remoção.

Diagrama Relacional



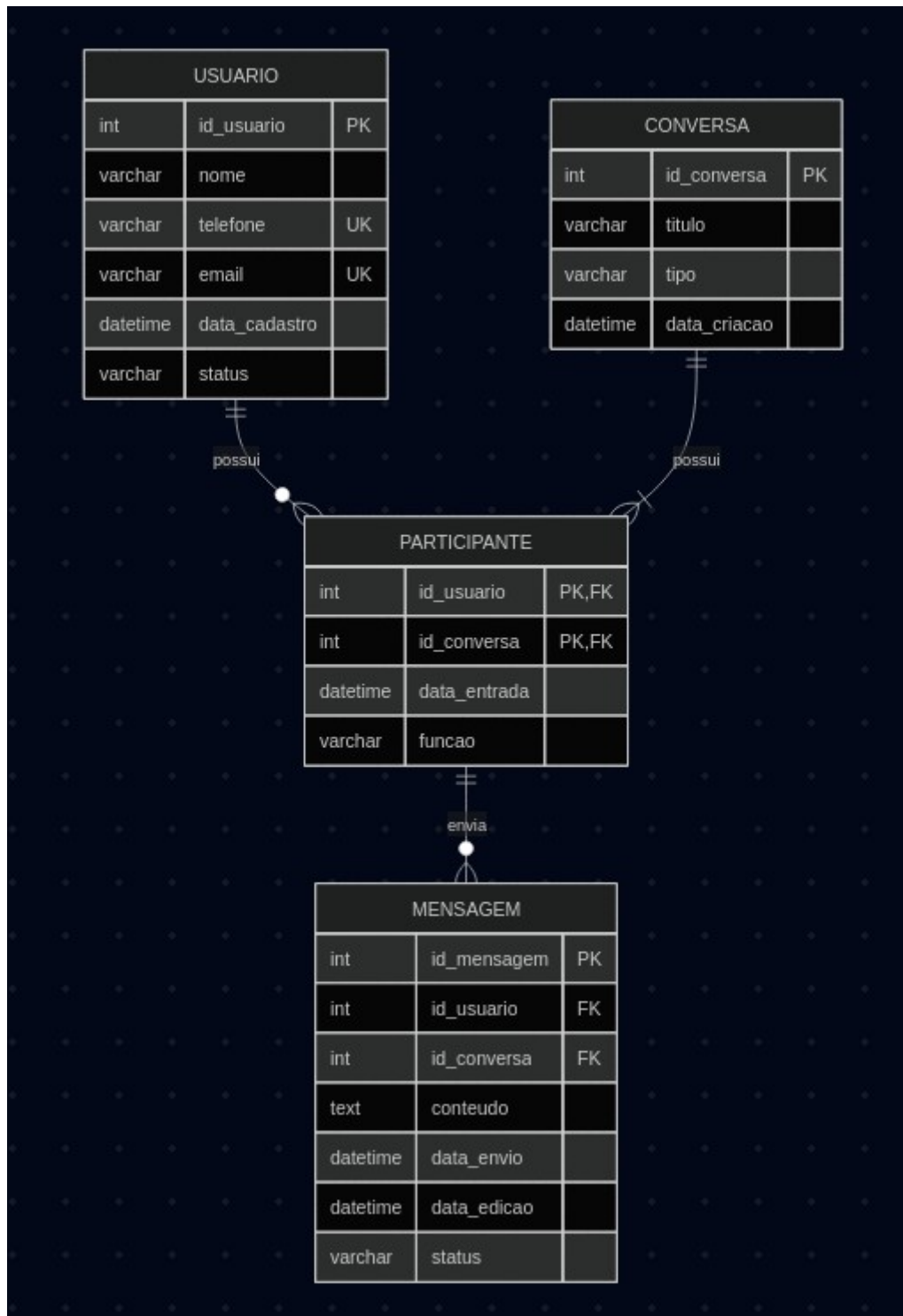
Fonte: Imagem elaborada pelo autor usando mermaid.live

PROJETO FÍSICO

O projeto físico corresponde à implementação do modelo lógico em um sistema gerenciador de banco de dados relacional. Nesta etapa, as tabelas, os atributos, as chaves e as restrições definidas anteriormente são convertidos em comandos SQL executáveis.

O banco de dados do Sistema de Mensageria Instantânea foi implementado utilizando MySQL, com tabelas organizadas para armazenar usuários, conversas, participantes e mensagens. Foram utilizadas chaves primárias, chaves estrangeiras, restrições de unicidade, validações de domínio e índices.

Diagrama Relacional do projeto físico



Fonte: Imagem elaborada pelo autor usando mermaid.live

O diagrama apresenta as tabelas USUARIO, CONVERSA, PARTICIPANTE e MENSAGEM, seus atributos, tipos de dados, chaves primárias, chaves estrangeiras e relacionamentos.

Criação do banco de dados

O banco de dados foi criado utilizando o conjunto de caracteres `utf8mb4`, permitindo o armazenamento de caracteres acentuados, símbolos e emojis.

```
CREATE DATABASE IF NOT EXISTS mensageria
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;
```

```
USE mensageria;
```

Criação das tabelas

Tabela USUARIO

A tabela USUARIO armazena as informações das pessoas cadastradas no sistema.

```
CREATE TABLE usuario (
    id_usuario INT NOT NULL AUTO_INCREMENT,
    nome VARCHAR(100) NOT NULL,
    telefone VARCHAR(20) NOT NULL,
    email VARCHAR(150),
    data_cadastro DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(10) NOT NULL DEFAULT 'ATIVO',

    CONSTRAINT pk_usuario
        PRIMARY KEY (id_usuario),

    CONSTRAINT uk_usuario_telefone
        UNIQUE (telefone),

    CONSTRAINT uk_usuario_email
        UNIQUE (email),

    CONSTRAINT ck_usuario_status
        CHECK (status IN ('ATIVO', 'INATIVO'))
) ENGINE = InnoDB;
```

Tabela CONVERSA

A tabela CONVERSA armazena as conversas privadas e em grupo.

```
CREATE TABLE conversa (
```

```

id_conversa INT NOT NULL AUTO_INCREMENT,
titulo VARCHAR(100),
tipo VARCHAR(10) NOT NULL,
data_criacao DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT pk_conversa
    PRIMARY KEY (id_conversa),

CONSTRAINT ck_conversa_tipo
    CHECK (tipo IN ('PRIVADA', 'GRUPO')),

CONSTRAINT ck_conversa_titulo
    CHECK (
        tipo = 'PRIVADA'
        OR titulo IS NOT NULL
    )
) ENGINE = InnoDB;

```

Tabela PARTICIPANTE

A tabela PARTICIPANTE representa a associação entre usuários e conversas.

```

CREATE TABLE participante (
    id_usuario INT NOT NULL,
    id_conversa INT NOT NULL,
    data_entrada DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    funcao VARCHAR(15) NOT NULL DEFAULT 'MEMBRO',

CONSTRAINT pk_participante
    PRIMARY KEY (id_usuario, id_conversa),

CONSTRAINT fk_participante_usuario
    FOREIGN KEY (id_usuario)
    REFERENCES usuario (id_usuario)
    ON UPDATE CASCADE
    ON DELETE RESTRICT,

CONSTRAINT fk_participante_conversa
    FOREIGN KEY (id_conversa)

```

```
REFERENCES conversa (id_conversa)
ON UPDATE CASCADE
ON DELETE CASCADE,
```

```
CONSTRAINT ck_participante_funcao
CHECK (
    funcao IN ('MEMBRO', 'ADMINISTRADOR')
)
) ENGINE = InnoDB;
```

Tabela MENSAGEM

A tabela MENSAGEM armazena os conteúdos enviados pelos usuários nas conversas.

```
CREATE TABLE mensagem (
    id_mensagem INT NOT NULL AUTO_INCREMENT,
    id_usuario INT NOT NULL,
    id_conversa INT NOT NULL,
    conteudo TEXT NOT NULL,
    data_envio DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    data_edicao DATETIME,
    status VARCHAR(10) NOT NULL DEFAULT 'ENVIADA',

    CONSTRAINT pk_mensagem
        PRIMARY KEY (id_mensagem),

    CONSTRAINT fk_mensagem_participante
        FOREIGN KEY (id_usuario, id_conversa)
        REFERENCES participante (id_usuario, id_conversa)
        ON UPDATE CASCADE
        ON DELETE RESTRICT,

    CONSTRAINT ck_mensagem_status
        CHECK (
            status IN ('ENVIADA', 'EDITADA', 'REMOVIDA')
        ),

    CONSTRAINT ck_mensagem_conteudo
        CHECK (CHAR_LENGTH(TRIM(conteudo)) > 0)
```

```
) ENGINE = InnoDB;
```

A chave estrangeira composta da tabela MENSAGEM garante que o autor da mensagem esteja cadastrado como participante da conversa.

Criação de índices

Foram criados índices adicionais para melhorar o desempenho das consultas mais frequentes.

```
CREATE INDEX idx_participante_conversa  
    ON participante (id_conversa);
```

```
CREATE INDEX idx_mensagem_conversa_data  
    ON mensagem (id_conversa, data_envio);
```

```
CREATE INDEX idx_mensagem_usuario  
    ON mensagem (id_usuario);
```

```
CREATE INDEX idx_usuario_nome  
    ON usuario (nome);
```

O índice criado sobre `id_conversa` e `data_envio` facilita a consulta do histórico de mensagens em ordem cronológica.

Inserção de dados de exemplo

Após a criação do banco de dados e de suas tabelas, foram inseridos dados de exemplo para permitir a validação da estrutura e a execução das operações do sistema.

Os registros inseridos representam usuários, conversas privadas e em grupo, participantes e mensagens.

Inserção de usuários

```
INSERT INTO usuario  
    (id_usuario, nome, telefone, email, status)  
VALUES  
    (  
        1,  
        'Ana Souza',  
        '479999990001',  
        'ana.souza@email.com',  
        'ATIVO'  
    ),  
    (  
        2,  
        'João Silva',  
        '321000000000',  
        'joao.silva@email.com',  
        'ATIVO'
```

```

2,
'Bruno Oliveira',
'479999990002',
'bruno.oliveira@email.com',
'ATIVO'
),
(
3,
'Carla Santos',
'479999990003',
'carla.santos@email.com',
'ATIVO'
),
(
4,
'Daniel Lima',
'479999990004',
'daniel.lima@email.com',
'ATIVO'
),
(
5,
'Eduarda Martins',
'479999990005',
NULL,
'INATIVO'
);

```

Inserção de conversas

```

INSERT INTO conversa
(id_conversa, titulo, tipo)
VALUES
(
1,
NULL,
'PRIVADA'
),

```



```
(
    2,
    'Grupo do Trabalho de Banco de Dados',
    'GRUPO'
),
(
    3,
    'Amigos da Faculdade',
    'GRUPO'
);
```

Inserção de participantes

```
INSERT INTO participante
    (id_usuario, id_conversa, funcao)
VALUES
    (1, 1, 'MEMBRO'),
    (2, 1, 'MEMBRO'),

    (1, 2, 'ADMINISTRADOR'),
    (2, 2, 'MEMBRO'),
    (3, 2, 'MEMBRO'),
    (4, 2, 'MEMBRO'),

    (2, 3, 'ADMINISTRADOR'),
    (3, 3, 'MEMBRO'),
    (4, 3, 'MEMBRO');
```

Inserção de mensagens

```
INSERT INTO mensagem
    (
        id_mensagem,
        id_usuario,
        id_conversa,
        conteudo,
        data_envio,
        data_edicao,
        status
    )
```

VALUES

```
(
  1,
  1,
  1,
  'Olá, Bruno! Tudo bem?',
  '2026-06-29 09:00:00',
  NULL,
  'ENVIADA'
),
(
  2,
  2,
  1,
  'Tudo bem! E com você?',
  '2026-06-29 09:01:00',
  NULL,
  'ENVIADA'
),
(
  3,
  1,
  2,
  'Criei o grupo para organizarmos o trabalho.',
  '2026-06-29 10:00:00',
  NULL,
  'ENVIADA'
),
(
  4,
  3,
  2,
  'Eu posso ajudar com o diagrama.',
  '2026-06-29 10:05:00',
  NULL,
  'ENVIADA'
```

```
),
(
    5,
    4,
    2,
    'Vou revisar o código SQL.',
    '2026-06-29 10:10:00',
    NULL,
    'ENVIADA'
),
(
    6,
    2,
    2,
    'Posso implementar o CRUD em Java.',
    '2026-06-29 10:15:00',
    '2026-06-29 10:16:00',
    'EDITADA'
),
(
    7,
    2,
    3,
    'Alguém quer estudar hoje?',
    '2026-06-29 11:00:00',
    NULL,
    'ENVIADA'
),
(
    8,
    3,
    3,
    'Mensagem removida',
    '2026-06-29 11:05:00',
    NULL,
    'REMOVIDA'
```

);

Os dados inseridos permitem testar diferentes situações do sistema, incluindo conversas privadas, grupos, administradores, membros, mensagens normais, mensagens editadas e mensagens removidas.